

Recursive Descent Parsing & LL(1) Predictive Parsing

FIRST • FOLLOW • Predictive Parsing Table • Complete parsing of $\text{id} + \text{id} * \text{id}$

Structured around the linked LL(1) non-recursive parsing lecture and the standard expression grammar used in compiler-design texts.

Compiler Design • 30-slide lecture deck

LL(1)

Left-to-right
Leftmost derivation
1 lookahead token

Learning outcomes

By the end, students should be able to construct and execute an LL(1) parser.

1

Explain recursive descent

One procedure per nonterminal; understand backtracking.

2

Transform a grammar

Remove left recursion and apply left factoring when needed.

3

Compute FIRST and FOLLOW

Apply the rules systematically, including ϵ cases.

4

Build the predictive table

Fill $M[A,a]$ using $FIRST(\alpha)$ and $FOLLOW(A)$.

5

Parse $id + id * id$

Trace stack, input, production/match, and acceptance.

6

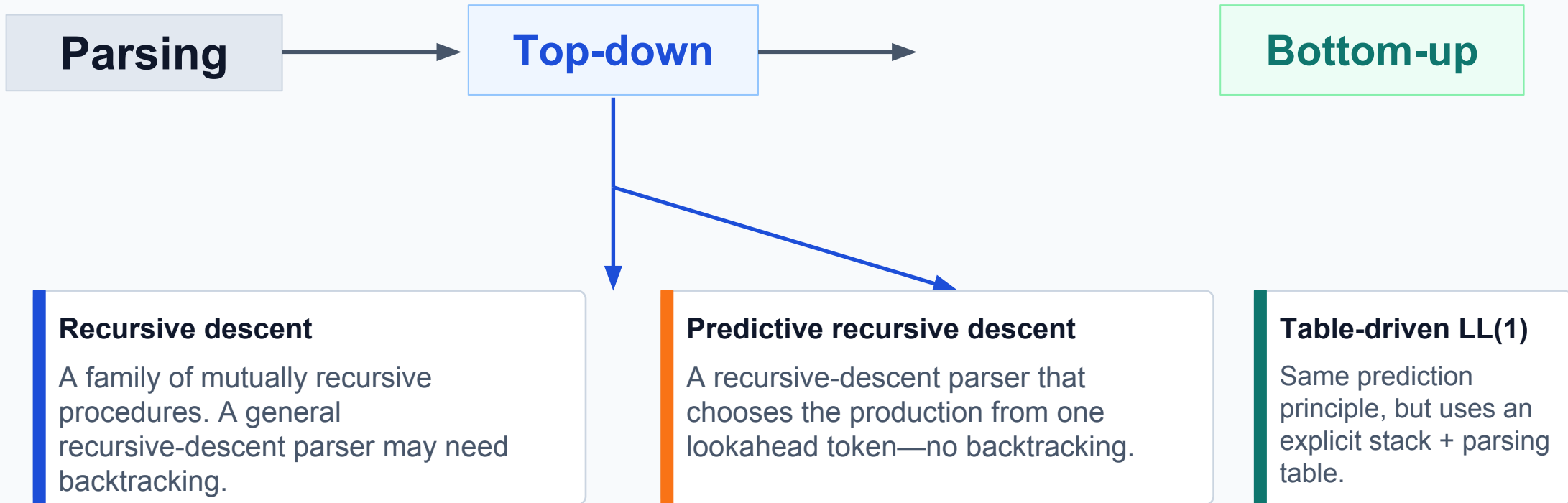
Write predictive procedures

Implement E , E' , T , T' , F without backtracking.

Where recursive descent fits

CONTEXT

Syntax analysis converts a token stream into a grammatical structure.



The linked video emphasizes the non-recursive (table-driven) LL(1) form; this deck connects it back to recursive descent.

Top-down parsing: core idea

TOP-DOWN

Start at the start symbol and repeatedly expand the leftmost nonterminal.

Expression grammar

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \varepsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Goal: derive the input from E

$\Rightarrow E$
 $\Rightarrow T E'$
 $\Rightarrow F T' E'$
 $\Rightarrow \text{id } T' E'$
 $\Rightarrow \text{id } E'$
 $\Rightarrow \text{id } + T E'$
 $\Rightarrow \text{id } + F T' E'$
 $\Rightarrow \text{id } + \text{id } T' E'$
 $\Rightarrow \text{id } + \text{id } * F T' E'$

id + id * id

A predictive parser performs this leftmost derivation deterministically using the next input token.

Recursive descent parser

RDP

One procedure is written for each nonterminal in the grammar.

Procedure mapping

$E \leftrightarrow \text{procedure } E()$
 $T \leftrightarrow \text{procedure } T()$
 $F \leftrightarrow \text{procedure } F()$

Mutual recursion

A procedure calls other nonterminal procedures according to the selected production.

Token matching

Terminals are matched against lookahead; input advances only after a successful match.

General form

```
procedure A() {  
  choose one production  $A \rightarrow X_1 X_2 \dots X_n$   
  for each  $X_i$ :  
    if  $X_i$  is a terminal: match( $X_i$ )  
    else: call  $X_i()$   
}
```

The difficult part is the highlighted phrase: which production should A choose?

General recursive descent may try one alternative, fail, restore the input position, and try another. Predictive parsing avoids that search.

Backtracking: why it is undesirable

BACKTRACK

If alternatives cannot be distinguished early, the parser may repeatedly undo work.

Example grammar

$$S \rightarrow a A c \mid a B d$$
$$A \rightarrow b$$
$$B \rightarrow b$$

Input: a b d \$

Predictive parsing asks: can the next token tell us the correct alternative immediately?

1

Try $S \rightarrow aAc$

Match a, match b via A, then expect c.

2

Failure

Lookahead is d, not c. The parser must restore the saved input position.

3

Try $S \rightarrow aBd$

Now the parse succeeds.

Predictive recursive descent

PREDICT

Use one lookahead token to select the correct production—no backtracking.

Prediction rule: choose $A \rightarrow \alpha$ when the lookahead belongs to the input-leading set for α ; if α can derive ϵ , FOLLOW(A) may trigger the ϵ -production.

FIRST tells us

Which terminals can appear first in strings derived from α ?

FOLLOW tells us

If $\alpha \Rightarrow^* \epsilon$, which terminals may legally appear after A ?

Parsing table stores

The chosen production for each pair [nonterminal, lookahead].

What LL(1) means

LL(1)

The linked lecture focuses on this deterministic top-down class.



Scan input

Left to right



Derivation

Leftmost derivation



Lookahead

One input token

An LL(1) parsing table must have at most one production in every cell.

Grammar prerequisites for predictive parsing

PREP

The grammar must allow a unique choice from one lookahead token.

1

Eliminate left recursion

$A \rightarrow A\alpha \mid \beta$ causes immediate recursion in a top-down parser.

2

Left factor when needed

If $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$, factor the common prefix α so the parser can delay the choice.

3

Compute FIRST and FOLLOW

These sets formalize what one-token lookahead can decide.

4

Check table conflicts

More than one production in $M[A,a]$ means the grammar is not LL(1) as written.

Important

Removing left recursion and left factoring do not magically make every grammar LL(1). They are grammar transformations that often help expose a deterministic predictive form.

Expression grammar before transformation

The natural arithmetic grammar is left recursive.

Left-recursive grammar

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Why recursive descent fails immediately

E() selects $E \rightarrow E + T$
→ calls E() again before consuming any input
→ calls E() again
→ ... infinite recursion

Therefore, eliminate left recursion before building a predictive parser.

Eliminating immediate left recursion

TRANSFORM

General transformation and application to E and T.

Pattern

$$A \rightarrow A \alpha \mid \beta$$

becomes

Equivalent form

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \varepsilon \end{aligned}$$

For expressions

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \varepsilon \end{aligned}$$

For terms

$$\begin{aligned} T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \varepsilon \end{aligned}$$

F remains $F \rightarrow (E) \mid id$ because it is not left recursive.

Final LL(1) expression grammar

REFERENCE

This grammar will be used for FIRST, FOLLOW, the table, and the parsing trace.

Grammar G

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \varepsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Nonterminals

$\{ E, E', T, T', F \}$

Terminals

$\{ \text{id}, +, *, (,) \}$

Start symbol

E

End marker

\$

FIRST: definition

FIRST

$\text{FIRST}(\alpha)$ tells us which terminal can begin a string derived from α .

$$\text{FIRST}(\alpha) = \{ \text{terminal } a \mid \alpha \Rightarrow^* a\beta \} \text{ plus } \varepsilon \text{ if } \alpha \Rightarrow^* \varepsilon$$

Terminal

$\text{FIRST}(\text{id}) = \{ \text{id} \}$
 $\text{FIRST}(+) = \{ + \}$

Nullable nonterminal

If $A \Rightarrow^* \varepsilon$, then $\varepsilon \in \text{FIRST}(A)$.

String $\alpha\beta$

If α can vanish, FIRST continues into β .

FIRST is about what can appear first—not what is written first syntactically in the production.

Rules for computing FIRST

FIRST RULES

Apply these until no set changes.

1

Terminal rule

If X is a terminal, $\text{FIRST}(X) = \{X\}$.

2

Direct ϵ rule

If $A \rightarrow \epsilon$, then $\epsilon \in \text{FIRST}(A)$.

3

Production $A \rightarrow X_1 X_2 \dots$

Add $\text{FIRST}(X_1) - \{\epsilon\}$. If X_1 is nullable, continue with X_2 , and so on.

4

All symbols nullable

If every X_i can derive ϵ , add ϵ to $\text{FIRST}(A)$.

Grammar used next

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \epsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \epsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

FIRST: step-by-step derivation

FIRST WORK

Start from productions whose first symbol is a terminal or whose production is ϵ .

1 From $F \rightarrow (E) \mid \text{id}$

$\text{FIRST}(F) = \{ (, \text{id} \}$

2 From $T' \rightarrow *FT' \mid \epsilon$

$\text{FIRST}(T') = \{ *, \epsilon \}$

3 From $T \rightarrow FT'$

F cannot derive ϵ , so $\text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$

4 From $E' \rightarrow +TE' \mid \epsilon$

$\text{FIRST}(E') = \{ +, \epsilon \}$

5 From $E \rightarrow TE'$

T cannot derive ϵ , so $\text{FIRST}(E) = \text{FIRST}(T) = \{ (, \text{id} \}$

No propagation past T or F is needed in $E \rightarrow TE'$ and $T \rightarrow FT'$ because T and F are not nullable.

FIRST sets: final result

FIRST TABLE

These sets drive the non- ϵ entries in the predictive parsing table.

Nonterminal	FIRST
E	{ id, (}
E'	{ +, ϵ }
T	{ id, (}
T'	{ *, ϵ }
F	{ id, (}

Immediate consequences

- E and T can begin only with id or (.
- E' has a real alternative beginning with +, or it may vanish.
- T' has a real alternative beginning with *, or it may vanish.
- The ϵ -productions require FOLLOW to know when vanishing is legal.

FOLLOW: definition

FOLLOW

FOLLOW(A) tells us which terminals may immediately follow A in a sentential form.

$$\text{FOLLOW}(A) = \{ \text{terminal } a \mid S \Rightarrow^* \alpha A a \beta \}$$

Start symbol

Place \$ in FOLLOW(S). Here $S = E$, so $\$ \in \text{FOLLOW}(E)$.

$A \rightarrow \alpha B \beta$

Add $\text{FIRST}(\beta) - \{\epsilon\}$ to FOLLOW(B).

$\beta \Rightarrow^* \epsilon$

Also add FOLLOW(A) to FOLLOW(B).

FOLLOW never contains ϵ .

Rules for computing FOLLOW

FOLLOW RULES

Repeatedly propagate symbols until no FOLLOW set changes.

1

Start symbol rule

$\$ \in \text{FOLLOW}(E)$ because E is the start symbol.

2

Terminal after B

For $A \rightarrow \alpha B a \beta$, add terminal a to $\text{FOLLOW}(B)$.

3

Nonterminal sequence

For $A \rightarrow \alpha B C \beta$, add $\text{FIRST}(C\beta) - \{\epsilon\}$ to $\text{FOLLOW}(B)$.

4

Suffix can vanish

If the suffix after B can derive ϵ , add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

5

B at production end

For $A \rightarrow \alpha B$, add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

Iteration matters

Some FOLLOW symbols arrive indirectly, so one pass may not be enough.

FOLLOW: step-by-step derivation

FOLLOW WORK

Use each production as a source of constraints.

Production	Constraint contributed
Start E	$\$ \in \text{FOLLOW}(E)$
$F \rightarrow (E)$	$) \in \text{FOLLOW}(E)$
$E \rightarrow T E'$	$\text{FIRST}(E') - \{\epsilon\} = \{+\} \subseteq \text{FOLLOW}(T)$; because E' nullable, $\text{FOLLOW}(E) \subseteq \text{FOLLOW}(T)$; $\text{FOLLOW}(E) \subseteq \text{FOLLOW}(E')$
$E' \rightarrow + T E'$	$\text{FIRST}(E') - \{\epsilon\} = \{+\} \subseteq \text{FOLLOW}(T)$; because E' nullable, $\text{FOLLOW}(E') \subseteq \text{FOLLOW}(T)$
$T \rightarrow F T'$	$\text{FIRST}(T') - \{\epsilon\} = \{*\} \subseteq \text{FOLLOW}(F)$; because T' nullable, $\text{FOLLOW}(T) \subseteq \text{FOLLOW}(F)$; $\text{FOLLOW}(T) \subseteq \text{FOLLOW}(T')$
$T' \rightarrow * F T'$	$\text{FIRST}(T') - \{\epsilon\} = \{*\} \subseteq \text{FOLLOW}(F)$; because T' nullable, $\text{FOLLOW}(T') \subseteq \text{FOLLOW}(F)$

After propagation stabilizes, collect duplicates and write each FOLLOW set once.

FOLLOW sets: final result

FOLLOW TABLE

These sets determine where ϵ -productions are entered in the table.

Nonterminal	FOLLOW
E	{), \$ }
E'	{), \$ }
T	{ +,), \$ }
T'	{ +,), \$ }
F	{ *, +,), \$ }

Why these matter for ϵ

- $E' \rightarrow \epsilon$ is used when lookahead is) or \$.
- $T' \rightarrow \epsilon$ is used when lookahead is +,), or \$.
- This is exactly how the parser decides that an expression or term has finished.
- FOLLOW provides “permission to stop” for nullable nonterminals.

LL(1) decision conditions

CHECK

Alternatives for a nonterminal must be distinguishable using one lookahead token.

For $A \rightarrow \alpha \mid \beta$: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta)$ must be empty. If $\epsilon \in \text{FIRST}(\alpha)$, then $\text{FIRST}(\beta) \cap \text{FOLLOW}(A)$ must also be empty (and vice versa).

Check E'

$\text{FIRST}(+TE') = \{+\}$; ϵ alternative uses
 $\text{FOLLOW}(E') = \{), \$\}$. No overlap $\rightarrow$ unique
decision.

Check T'

$\text{FIRST}(*FT') = \{*\}$; ϵ alternative uses
 $\text{FOLLOW}(T') = \{+,), \$\}$. No overlap $\rightarrow$ unique
decision.

Therefore the standard expression grammar is LL(1).

Algorithm to construct the predictive parsing table

TABLE RULE

For every production $A \rightarrow \alpha$, fill $M[A,a]$ using FIRST and FOLLOW.

1

Use FIRST(α)

For each terminal a in $\text{FIRST}(\alpha) - \{\epsilon\}$, place $A \rightarrow \alpha$ in $M[A,a]$.

2

If α can derive ϵ

For each b in $\text{FOLLOW}(A)$, place $A \rightarrow \alpha$ in $M[A,b]$.

3

If \$ is in FOLLOW(A)

\$ is just another table column; the same ϵ rule applies.

4

All other cells

Mark them error / blank. The parser reports a syntax error if it consults such a cell.

LL(1) test by table

If any cell receives two different productions, the grammar is not LL(1) as written.

Filling the table: production by production

TABLE BUILD

This is the crucial bridge from FIRST/FOLLOW to parsing actions.

Production	FIRST(α)	FOLLOW used?	Table entries
$E \rightarrow T E'$	{ id, (}	No	$M[E, id], M[E, (] = E \rightarrow T E'$
$E' \rightarrow + T E'$	{ + }	No	$M[E', +] = E' \rightarrow + T E'$
$E' \rightarrow \epsilon$	{ ϵ }	$FOLLOW(E') = \{), \$\}$	$M[E',), M[E', \$] = E' \rightarrow \epsilon$
$T \rightarrow F T'$	{ id, (}	No	$M[T, id], M[T, (] = T \rightarrow F T'$
$T' \rightarrow * F T'$	{ * }	No	$M[T', *] = T' \rightarrow * F T'$
$T' \rightarrow \epsilon$	{ ϵ }	$FOLLOW(T') = \{+,), \$\}$	$M[T', +], M[T',), M[T', \$] = T' \rightarrow \epsilon$
$F \rightarrow (E)$	{ (}	No	$M[F, (] = F \rightarrow (E)$
$F \rightarrow id$	{ id }	No	$M[F, id] = F \rightarrow id$

Notice that FOLLOW is consulted only for nullable right-hand sides.

Complete LL(1) predictive parsing table

FINAL TABLE

Rows are nonterminals; columns are the lookahead token.

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Blank cell = syntax error

Non-recursive predictive parser: architecture

The video's parser uses an explicit stack and a two-dimensional parsing table.

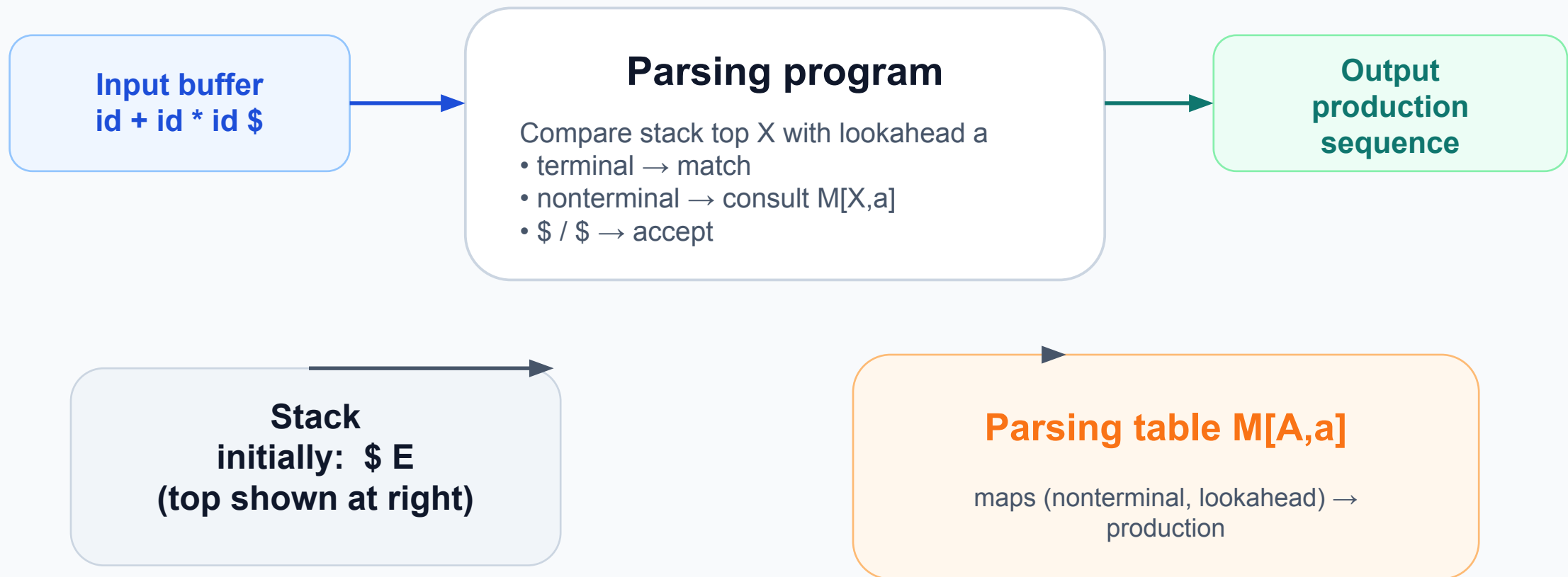


Table-driven LL(1) parsing algorithm

ALGORITHM

At each iteration, inspect the stack top X and current lookahead a .

Initialize: stack = \$ E, input = w \$

Condition	Action
$X = a = \$$	Accept
X is a terminal and $X = a$	Pop X ; advance input to next token
X is a terminal and $X \neq a$	Syntax error
X is a nonterminal and $M[X,a] = X \rightarrow Y_1 \dots Y_n$	Pop X ; push $Y_n \dots Y_1$ so Y_1 becomes the new top
X is a nonterminal and $M[X,a]$ is blank	Syntax error

In the following trace, the rightmost stack symbol is the top of stack.

Parsing $\text{id} + \text{id} * \text{id}$ — trace (Part 1)

TRACE 1

Start from stack $\$ E$ and repeatedly use the parsing table.

Step	Stack (top $\rightarrow$ right)	Remaining input	Action
0	$\$ E$	$\text{id} + \text{id} * \text{id} \$$	$M[E, \text{id}] = E \rightarrow TE'$
1	$\$ E' T$	$\text{id} + \text{id} * \text{id} \$$	$M[T, \text{id}] = T \rightarrow FT'$
2	$\$ E' T' F$	$\text{id} + \text{id} * \text{id} \$$	$M[F, \text{id}] = F \rightarrow \text{id}$
3	$\$ E' T' \text{id}$	$\text{id} + \text{id} * \text{id} \$$	match id
4	$\$ E' T'$	$+ \text{id} * \text{id} \$$	$M[T', +] = T' \rightarrow \epsilon$
5	$\$ E'$	$+ \text{id} * \text{id} \$$	$M[E', +] = E' \rightarrow +TE'$
6	$\$ E' T +$	$+ \text{id} * \text{id} \$$	match $+$
7	$\$ E' T$	$\text{id} * \text{id} \$$	$M[T, \text{id}] = T \rightarrow FT'$
8	$\$ E' T' F$	$\text{id} * \text{id} \$$	$M[F, \text{id}] = F \rightarrow \text{id}$

The ϵ move at step 4 says: the first term has ended because $+$ belongs to $\text{FOLLOW}(T')$.

Parsing $\text{id} + \text{id} * \text{id}$ — trace (Part 2)

TRACE 2

Multiplication is recognized inside the second term before the expression ends.

Step	Stack (top $\rightarrow$ right)	Remaining input	Action
9	\$ E' T' id	id * id \$	match id
10	\$ E' T'	* id \$	$M[T', *] = T' \rightarrow *FT'$
11	\$ E' T' F *	* id \$	match *
12	\$ E' T' F	id \$	$M[F, \text{id}] = F \rightarrow \text{id}$
13	\$ E' T' id	id \$	match id
14	\$ E' T'	\$	$M[T', \$] = T' \rightarrow \epsilon$
15	\$ E'	\$	$M[E', \$] = E' \rightarrow \epsilon$
16	\$	\$	ACCEPT

Accepted: $\text{id} + \text{id} * \text{id}$ is generated by the grammar

Equivalent predictive recursive-descent procedures

[CODE](#)

The same FIRST/FOLLOW decisions can be encoded directly as procedures.

```
E() { T(); Eprime(); }

Eprime() {
  if (lookahead == '+') {
    match('+'); T(); Eprime();
  } else if (lookahead == ')' || lookahead == '$') {
    return;          //  $\epsilon$ 
  } else error();
}
```

```
T() { F(); Tprime(); }

Tprime() {
  if (lookahead == '*') {
    match('*'); F(); Tprime();
  } else if (lookahead == '+' || lookahead == ')' || lookahead == '$') {
    return;          //  $\epsilon$ 
  } else error();
}

F() {
  if (lookahead == 'id') match('id');
  else if (lookahead == '(') { match('('); E(); match(')'); }
  else error();
}
```

The conditions for ϵ in E' and T' are exactly $FOLLOW(E')$ and $FOLLOW(T')$.

Summary and references

WRAP-UP

A complete workflow from grammar to executable predictive parsing.

1

Prepare grammar

Eliminate left recursion; left factor if required.

2

Compute FIRST

Know which token can begin each alternative.

3

Compute FOLLOW

Know when nullable nonterminals may disappear.

4

Build $M[A,a]$

Use FIRST for ordinary alternatives and FOLLOW for ϵ .

5

Execute parser

Use stack + lookahead + table or equivalent recursive procedures.

6

Validate

$\text{id} + \text{id} * \text{id}$ is accepted by the completed LL(1) table.

References

- Linked YouTube lecture: “LL(1) Parser in Compiler Design | Non-Recursive Parsing | Example 1”
- Alfred V. Aho, Ravi Sethi, Jeffrey D. Ullman, Compilers: Principles, Techniques, and Tools (classic Dragon Book)
- Standard LL(1) expression grammar used throughout compiler-design teaching

Key idea: FIRST predicts how to start; FOLLOW tells nullable nonterminals when to stop.